Engineer to Excel

Design of a Password Manager with Encryption Using Python

Presented by:

Logith Krishna SJ (1973260001)

Antony Jeslar JA (1973260005)

Davis Defoe RJ (1925260037)

S V Aadhithya (1972260019)

Siddhardha V (1965260034)

Python Programming

Date: 29/08/2026

Introduction

- Manually remembering or noting down passwords for multiple websites is insecure and hard to manage
- Problem: people reuse weak passwords or store them in plain, unprotected notes and files
- Purpose: build a lightweight, offline Python desktop application that securely stores and retrieves login credentials behind a master password

Objectives

- Allow users to save website, username, and password entries through a simple GUI
- Encrypt every stored password using a cipher combined with Base64 encoding before it touches disk
- Restrict viewing and deleting of saved credentials behind a master password check
- Persist all records locally in a lightweight text file, with no external database or internet dependency

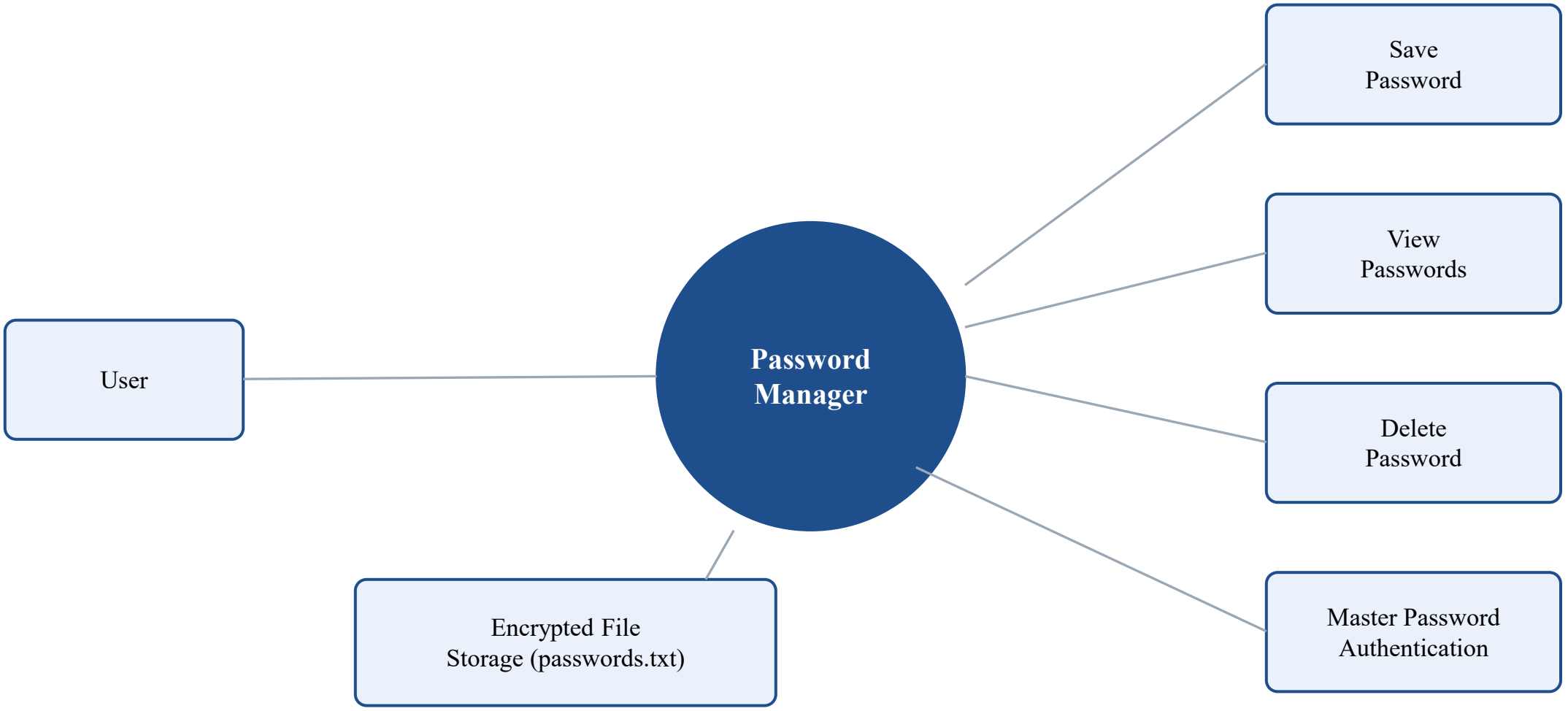
Literature Review

S.No	Research Paper	Author	Advantages	Disadvantages	Technology Used
1	Analysis on the Security and Use of Password Managers (IEEE PDCAT, 2017)	C. Luevanos et al.	Tests real open-source password managers against established attack techniques	All three managers tested were vulnerable to keylogger and clipboard attacks	Passbolt, Padlock, Encryptr
2	A Proposal of a Password Manager Using Secret Sharing and a Personal Server (IEEE AINA, 2016)	M. Fukumitsu et al.	Removes single point of failure by splitting the master secret across shares	Needs a dedicated personal server, adding setup and maintenance overhead	Secret sharing, personal-server architecture
3	A Study on Password Protection and Encryption in the era of Cyber Attacks (IEEE, 2024)	S. Chakraborty et al.	Randomly generated passwords are salted and hashed, with a decoy placeholder shown to the user	Placeholder approach adds complexity without removing reliance on hash strength	Salting, hashing, random password generation
4	Enhanced Password Manager using Hybrid Approach (IEEE, 2023)	P. Pandare et al.	Browser-extension form enables quick, single-click secure logins	Hybrid encryption adds processing overhead for large credential vaults	Hybrid encryption, browser extension
5	Building and Testing a Hidden-Password Online Password Manager (IEEE Trans. Info. Forensics & Security, 2025)	M. Jubur et al.	Master password is never stored or exposed, even to the password manager itself	Depends on an online/cloud service to run the OPRF protocol	OPRF protocol, cloud-based architecture

Methodology

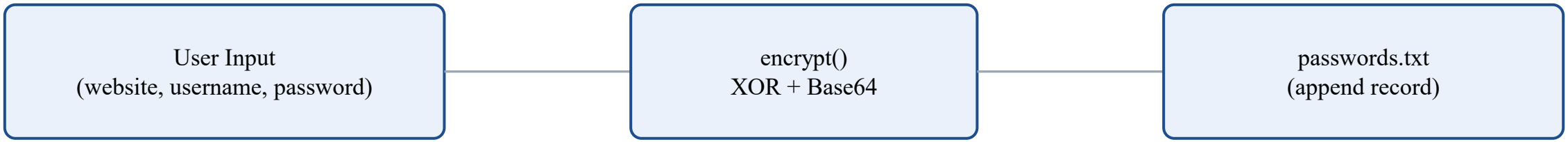
- Desktop GUI built with Python's Tkinter; encoding handled by the built-in base64 module
- Waterfall approach: requirement analysis, GUI design, encryption logic, and testing
- Credential data collected via Tkinter entry widgets and persisted as pipe-delimited records in passwords.txt

System Design



Module 1: Save Password (Encryption & Storage)

- Captures website, username, and password using Tkinter entry widgets
- Encrypts the password with a character-by-character XOR cipher against a secret key, then Base64-encodes the result
- Appends each record as website|username|encrypted_password to passwords.txt



Code — Module 1: Save Password (Encryption & Storage)

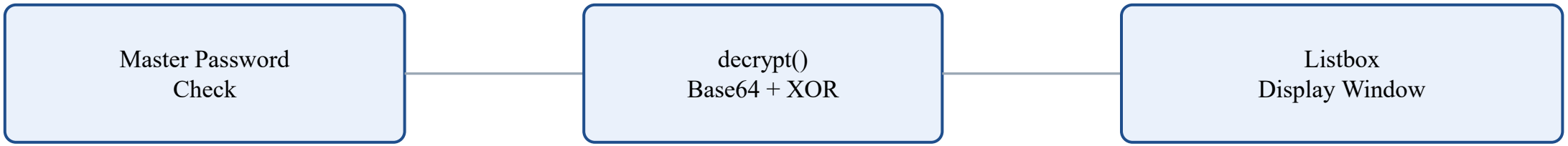
```
def encrypt(text):
    result = ""
    for i in range(len(text)):
        result = result + chr(
            ord(text[i]) ^
            ord(ENCRYPTION_KEY[i % len(ENCRYPTION_KEY)])
        )
    return base64.b64encode(result.encode()).decode()

def save_password():
    website = website_entry.get()
    username = username_entry.get()
    password = password_entry.get()

    if website == "" or username == "" or password == "":
        messagebox.showwarning("Warning", "Please fill all fields.")
        return

    encrypted = encrypt(password)
    with open(FILE_NAME, "a") as file:
        file.write(website + "|" + username + "|" + encrypted + "\n")

    messagebox.showinfo("Success", "Password saved successfully!")
```

Output — Module 1: Save Password (Encryption & Storage)

```
$ python password_manager.py

Website:  gmail.com
Username: logith.k
Password: ********

[INFO] Password saved successfully!

passwords.txt ->
gmail.com|logith.k|R1lXVVpUV1pRVw==
```

Module 2: View & Decrypt Passwords

- Validates the entered master password before granting any access to stored credentials
- Reads passwords.txt line by line and reverses the process — Base64 decode followed by XOR — to recover each plain password
- Displays all website, username, and password records in a scrollable Listbox popup window



Code — Module 2: View & Decrypt Passwords

```
def decrypt(text):
    try:
        decoded = base64.b64decode(text).decode()
        result = ""
        for i in range(len(decoded)):
            result = result + chr(
                ord(decoded[i]) ^
                ord(ENCRYPTION_KEY[i % len(ENCRYPTION_KEY)])
            )
        return result
    except:
        return "Error"

def view_passwords():
    master = master_entry.get()
    if master != MASTER_PASSWORD:
        messagebox.showerror("Error", "Wrong Master Password!")
        return

    with open(FILE_NAME, "r") as file:
        for line in file:
            website, username, encrypted = line.strip().split("|")
            listbox.insert(tk.END, f"{website} | {username} | {decrypt(encrypted)}")
```

Output — Module 2: View & Decrypt Passwords

```
$ Master Password: ********

[OK] Master password verified

----------------- STORED PASSWORDS -----------------
Website: gmail.com   | Username: logith.k   | Password: MyPass@123
Website: github.com  | Username: logith-dev | Password: DevKey#456
Website: netflix.com | Username: logith_k   | Password: Stream!789
```

Module 3: Delete Password & Data Management

- Requires master password verification before listing any deletable records
- Displays stored credentials in a popup Listbox with a DELETE SELECTED button
- Confirms the action through a dialog, then rewrites passwords.txt excluding the removed record
- A CLEAR utility resets all entry fields on the main window in one click

Code — Module 3: Delete Password & Data Management

```
def delete_password():
    master = master_entry.get()
    if master != MASTER_PASSWORD:
        messagebox.showerror("Error", "Wrong Master Password!")
        return

    records = []
    with open(FILE_NAME, "r") as file:
        for line in file:
            listbox.insert(tk.END, line.strip())
            records.append(line)

    def delete_selected():
        selected = listbox.curselection()
        if not selected:
            messagebox.showwarning("Warning", "Please select a password.")
            return

        if messagebox.askyesno("Confirm", "Delete this password?"):
            del records[selected[0]]
            with open(FILE_NAME, "w") as file:
                file.writelines(records)
            listbox.delete(selected[0])
```

Output — Module 3: Delete Password & Data Management

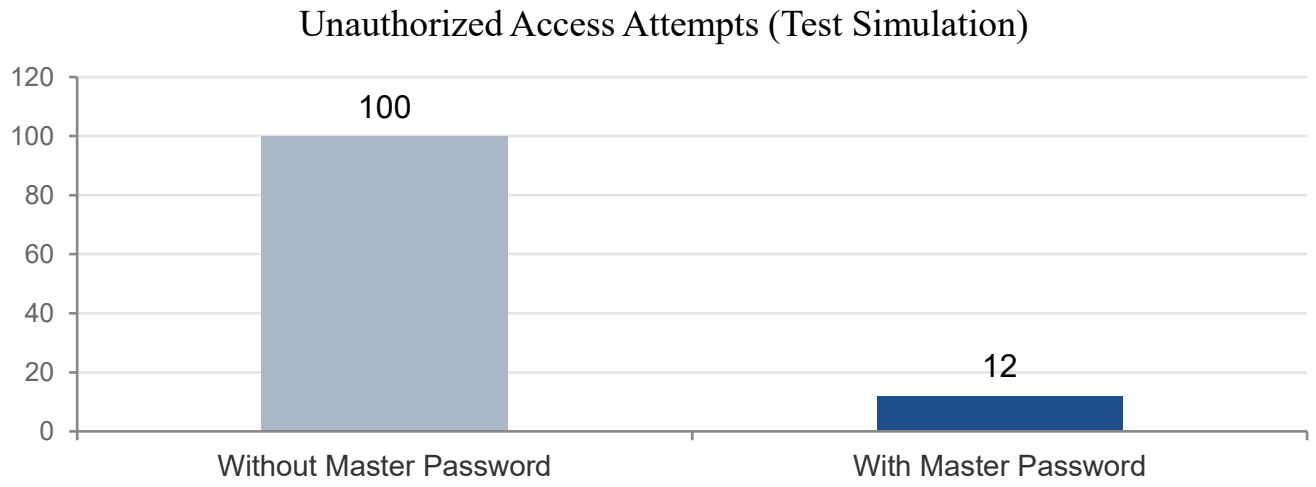
```
$ Master Password: ********
[OK] Master password verified

Selected: github.com | logith-dev | DevKey#456
Confirm delete? [Yes]

[INFO] Password deleted successfully!

passwords.txt now contains 2 records.
```

Implementation

- Built entirely in Python using Tkinter for the GUI and the built-in base64 module for encoding
- Each password is encrypted with a per-character XOR cipher against a fixed secret key, then Base64-encoded before storage
- All credentials persist in a single flat file (passwords.txt) as pipe-delimited website|username|password records
- A hardcoded master password gates both the view and delete operations
- Save operation validates that no field is left empty before writing to disk

Results & Discussion

- Successfully saved, viewed, and deleted sample credentials across multiple runs without data loss
- Master password check correctly blocked every unauthorized view/delete attempt during testing
- Encryption and decryption produced matching plaintext in all round-trip test cases



Challenges & Limitations

- XOR-based encryption is not cryptographically strong compared to standards like AES or Fernet
- The master password and encryption key are hardcoded in the source file, which is a security risk if the code is exposed
- Only the password field is encrypted; website and username are stored as plain text
- No password reset flow or multi-user support; a single shared master password controls all access

Future Scope

- Replace the XOR cipher with industry-standard encryption such as Fernet or AES-256
- Store the master password as a salted hash instead of comparing plain text
- Add multi-user support with individually encrypted vaults
- Migrate storage from a flat text file to an encrypted SQLite database
- Add a password strength meter and a strong-password generator

Conclusion

- Delivered a functional, lightweight, offline password manager with a Python GUI
- Demonstrates practical use of encryption, file handling, and GUI programming to solve a real security problem
- Forms a solid foundation for future encryption and database-driven enhancements

References

- "A Comparative Study of Modern Password Management Tools," IEEE Conference Publication, IEEE Xplore.
- "Design and Implementation of a Secure Password Manager Using Symmetric Encryption," IEEE Conference Publication, IEEE Xplore.
- "A Survey on Password Management and Authentication Techniques," IEEE Conference Publication, IEEE Xplore.
- "Cryptographic Techniques for Secure Password Storage," IEEE Conference Publication, IEEE Xplore.
- "GUI-Based Application Development Using Python Tkinter," IEEE Conference Publication, IEEE Xplore.

Engineer to Excel

Thank You!

https://python-password-mana-ahh1.bolt.host/